

What is Exception in Java?

Dictionary Meaning: Exception is an abnormal condition.

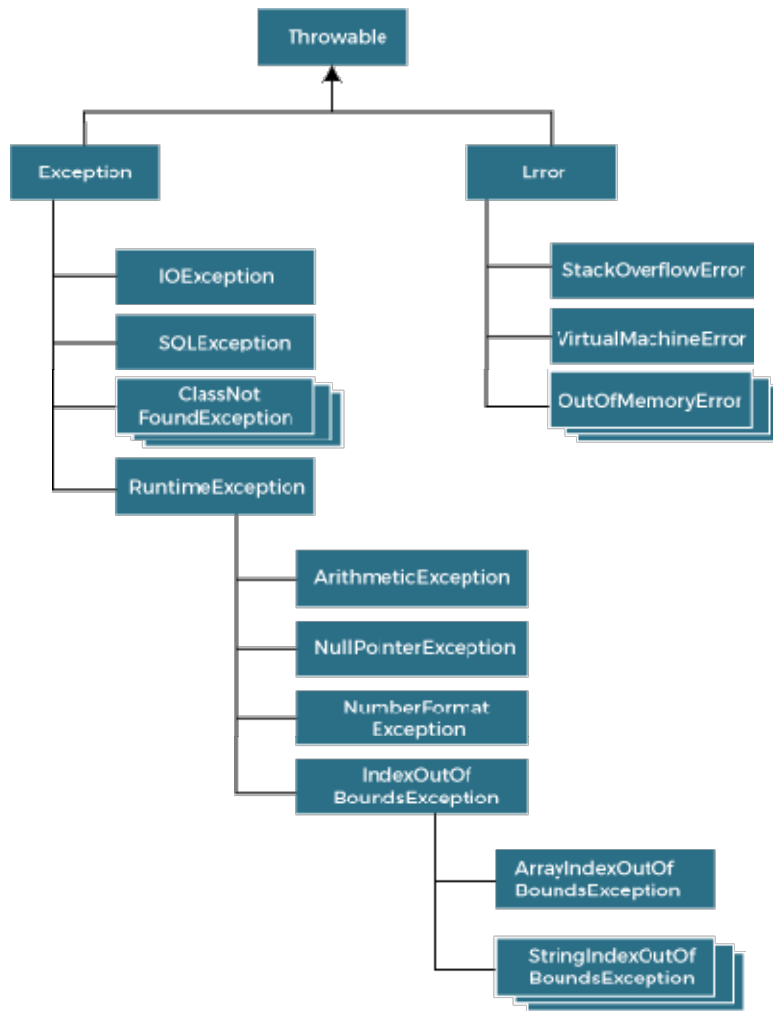
In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

What is Exception Handling?

Exception Handling is a mechanism to handle runtime errors such as `ClassNotFoundException`, `IOException`, `SQLException`, `RemoteException`, etc.

Hierarchy of Java Exception classes

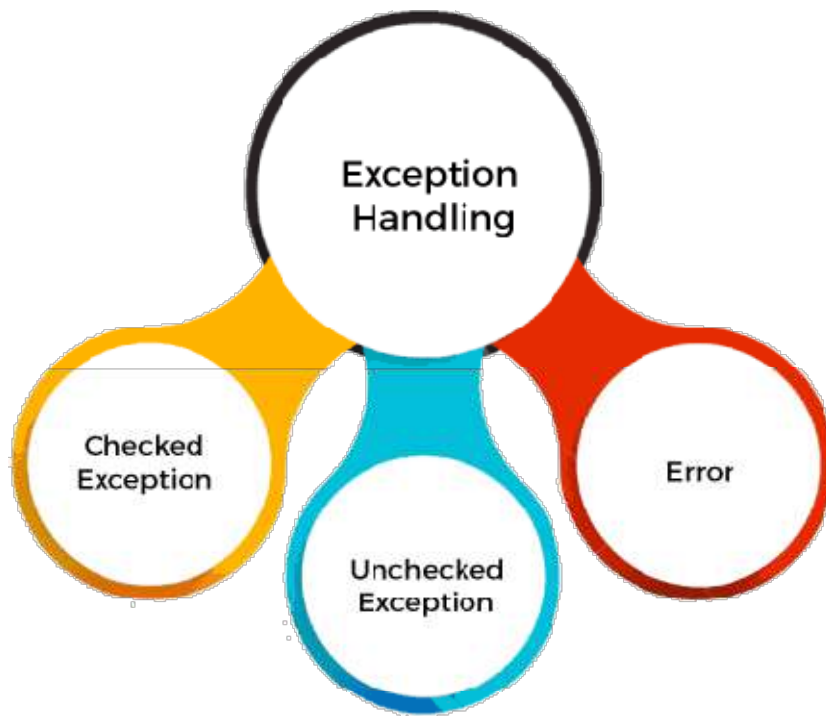
The `java.lang.Throwable` class is the root class of Java Exception hierarchy inherited by two subclasses: `Exception` and `Error`. The hierarchy of Java Exception classes is given below:



Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

1. Checked Exception
2. Unchecked Exception
3. Error



Difference between Checked and Unchecked Exceptions

1) Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java Exception Keywords

Java provides five keywords that are used to handle the exception. The following table describes each.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Java Exception Handling Example

Let's see an example of Java Exception Handling in which we are using a try-catch statement to handle the exception.

JavaExceptionExample.java

```

1. public class JavaExceptionExample{
2.     public static void main(String args[]){
3.         try{
4.             //code that may raise exception
5.             int data=100/0;
6.         }catch(ArithmeticException e){System.out.println(e);}
7.         //rest code of the program
8.         System.out.println("rest of the code...");
9.     }
10. }
```

Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zero
rest of the code...
```

Common Scenarios of Java Exceptions

There are given some scenarios where unchecked exceptions may occur. They are as follows:

1) A scenario where `ArithmeticException` occurs

If we divide any number by zero, there occurs an `ArithmeticException`.

```
1. int a=50/0;//ArithmeticException
```

2) A scenario where `NullPointerException` occurs

If we have a null value in any variable, performing any operation on the variable throws a `NullPointerException`.

```
1. String s=null;
2. System.out.println(s.length());//NullPointerException
```

3) A scenario where `NumberFormatException` occurs

If the formatting of any variable or number is mismatched, it may result into `NumberFormatException`. Suppose we have a string variable that has characters; converting this variable into digit will cause `NumberFormatException`.

```
1. String s="abc";
2. int i=Integer.parseInt(s);//NumberFormatException
```

4) A scenario where `ArrayIndexOutOfBoundsException` occurs

When an array exceeds to its size, the `ArrayIndexOutOfBoundsException` occurs. There may be other reasons to occur `ArrayIndexOutOfBoundsException`. Consider the following statements.

```
1. int a[]=new int[5];
2. a[10]=50; //ArrayIndexOutOfBoundsException
```

Java throw Exception

We can also define our own set of conditions and throw an exception explicitly using `throw` keyword. For example, we can throw `ArithmeticException` if we divide a number by another number. Here, we just need to set the condition and throw exception using `throw` keyword.

The syntax of the Java `throw` keyword is given below.

`throw` Instance i.e.,

```
. throw new exception_class("error message");
```

Let's see the example of throw IOException.

```
. throw new IOException("sorry device error");
```

Where the Instance must be of type Throwable or subclass of Throwable. For example, Exception is the sub class of Throwable and the user-defined exceptions usually extend the Exception class.

Java throw keyword Example

Example 1: Throwing Unchecked Exception

In this example, we have created a method named validate() that accepts an integer as a parameter. If the age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote.

TestThrow1.java

In this example, we have created the validate method that takes integer value as a parameter. If the age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote.

```
. public class TestThrow1 {
2.     //function to check if person is eligible to vote or not
3.     public static void validate(int age) {
4.         if(age<18) {
5.             //throw Arithmetic exception if not eligible to vote
6.             throw new ArithmeticException("Person is not eligible to vote");
7.         }
8.         else {
9.             System.out.println("Person is eligible to vote!!");
10.        }
11.    }
12.    //main method
13.    public static void main(String args[]){
14.        //calling the function
15.        validate(13);
16.        System.out.println("rest of the code...");
17.    }
18. }
```

Output:

```

C:\Users\Anurati\Desktop\abcDemo>javac TestThrow1.java

C:\Users\Anurati\Desktop\abcDemo>java TestThrow1
Exception in thread "main" java.lang.ArithmeticException: Person is not eligible to
vote
    at TestThrow1.validate(TestThrow1.java:8)
    at TestThrow1.main(TestThrow1.java:18)

```

The above code throw an unchecked exception. Similarly, we can also throw unchecked and user defined exceptions.

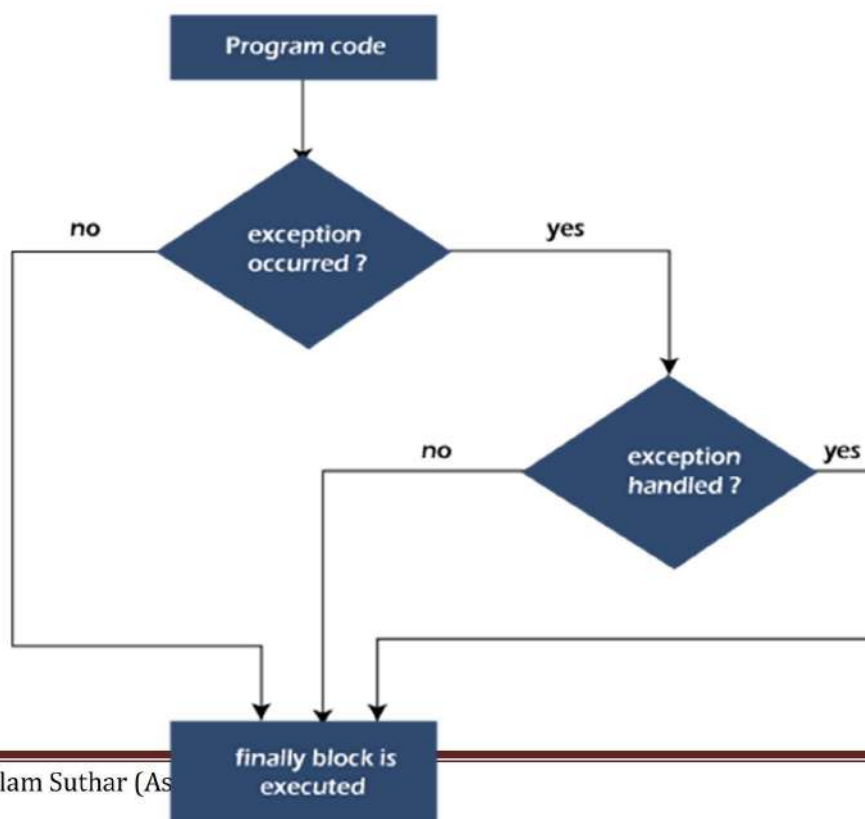
Note: If we throw unchecked exception from a method, it is must to handle the exception or declare in throws clause.

If we throw a checked exception using throw keyword, it is must to handle the exception using catch block or the method must declare it using throws declaration.

Java finally block

Java finally block is a block used to execute important code such as closing the connection, etc.

Java finally block is always executed whether an exception is handled or not. Therefore, it contains all the necessary statements that need to be printed regardless of the exception occurs or not.



The finally block follows the try-catch block.

Flowchart of finally block

Note: If you don't handle the exception, before terminating the program, JVM executes finally block (if any).

Why use Java finally block?

- finally block in Java can be used to put "**cleanup**" code such as closing a file, closing connection, etc.
- The important statements to be printed can be placed in the finally block.

Usage of Java finally

Let's see the different cases where Java finally block can be used.

Case 1: When an exception does not occur

Let's see the below example where the Java program does not throw any exception, and the finally block is executed after the try block.

TestFinallyBlock.java

```

1. class TestFinallyBlock {
2.     public static void main(String args[]){
3.         try{
4.             //below code do not throw any exception
5.             int data=25/5;
6.             System.out.println(data);
7.         }
8.         //catch won't be executed
9.         catch(NullPointerException e){
10.            System.out.println(e);
11.        }
12.        //executed regardless of exception occurred or not
13.        finally {
14.            System.out.println("finally block is always executed");
15.        }
16.
17.        System.out.println("rest of phe code...");
18.    }
19. }

```


Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestFinallyBlock.java
C:\Users\Anurati\Desktop\abcDemo>java TestFinallyBlock
5
finally block is always executed
rest of the code...
```

Case 2: When an exception occur but not handled by the catch block

Let's see the the following example. Here, the code throws an exception however the catch block cannot handle it. Despite this, the finally block is executed after the try block and then the program terminates abnormally.

TestFinallyBlock1.java

```
1. public class TestFinallyBlock1{
2.     public static void main(String args[]){
3.
4.         Try
5.         {
6.
7.             System.out.println("Inside the try block");
8.
9.             //below code throws divide by zero exception
10.            int data=25/0;
11.            System.out.println(data);
12.        }
13.        //cannot handle Arithmetic type exception
14.        //can only accept Null Pointer type exception
15.        catch(NullPointerException e
16. ){
17.            System.out.println(e);
18.        }
19.
20.        //executes regardless of exception occurred or not
21.        Finally
22.        {
```

```
23.     System.out.println("finally block is always executed");
24. }
25.
26.     System.out.println("rest of the code...");
27. }
28. }
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestFinallyBlock1.java

C:\Users\Anurati\Desktop\abcDemo>java TestFinallyBlock1
Inside the try block
finally block is always executed
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at TestFinallyBlock1.main(TestFinallyBlock1.java:9)
```

Graphics Class

The Graphics class is the abstract super class for all graphics contexts which allow an application to draw onto components that can be realized on various devices, or onto off-screen images as well.

Java Applet

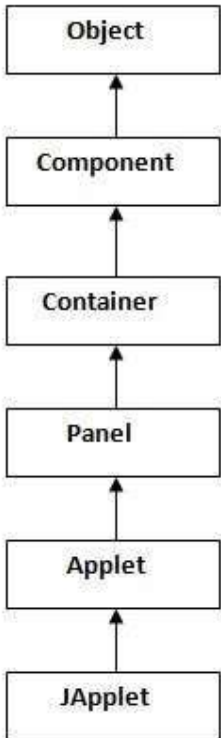
Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

Advantage of Applet

There are many advantages of applet. They are as follows:

- It works at client side so less response time.
- Secured
- It can be executed by browsers running under many platforms, including Linux, Windows, Mac Os etc.
-

Hierarchy of Applet



As displayed in the above diagram, Applet class extends Panel. Panel class extends Container which is the subclass of Component.

Lifecycle of Java Applet

1. Applet is initialized.
2. Applet is started.
3. Applet is painted.
4. Applet is stopped.
5. Applet is destroyed.

Applet Lifecycle



Lifecycle methods for Applet:

The `java.applet.Applet` class provides 4 life cycle methods and `java.awt.Component` class provides 1 life cycle method for an applet.

`java.applet.Applet` class

For creating any applet `java.applet.Applet` class must be inherited. It provides 4 life cycle methods of applet.

1. **`public void init():`** is used to initialize the Applet. It is invoked only once.
2. **`public void start():`** is invoked after the `init()` method or browser is maximized. It is used to start the Applet.
3. **`public void stop():`** is used to stop the Applet. It is invoked when Applet is stop or browser is minimized.
4. **`public void destroy():`** is used to destroy the Applet. It is invoked only once.

`java.awt.Component` class

The Component class provides 1 life cycle method of applet.

1. **`public void paint(Graphics g):`** is used to paint the Applet. It provides Graphics class object that can be used for drawing oval, rectangle, arc etc.

Who is responsible to manage the life cycle of an applet?

Java Plug-in software.

How to run an Applet?

There are two ways to run an applet

1. By html file.
2. By appletViewer tool (for testing purpose).

Simple example of Applet by html file:

To execute the applet by html file, create an applet and compile it. After that create an html file and place the applet code in html file. Now click the html file.

```
1. //First.java
2. import java.applet.Applet;
3. import java.awt.Graphics;
4. public class First extends Applet{
5.
6.     public void paint(Graphics g){
7.         g.drawString("welcome",150,150);
8.     }
9.
10. }
```

Note: class must be public because its object is created by Java Plugin software that resides on the browser.

myapplet.html

```
1. <html>
2. <body>
3. <applet code="First.class" width="300" height="300">
4. </applet>
5. </body>
6. </html>
```

Simple example of Applet by appletviewer tool:

To execute the applet by appletviewer tool, create an applet that contains applet tag in comment and compile it. After that run it by: appletviewer First.java. Now Html file is not required but it is for testing purpose only.

```

1. //First.java
2. import java.applet.Applet;
3. import java.awt.Graphics;
4. public class First extends Applet{
5.
6. public void paint(Graphics g){
7. g.drawString("welcome to applet",150,150);
8. }
9.
10. }
11. /*
12. <applet code="First.class" width="300" height="300">
13. </applet>
14. */

```

To execute the applet by appletviewer tool, write in command prompt:

```

c:\>javac First.java
c:\>appletviewer First.java

```

Displaying Graphics in Applet

java.awt.Graphics class provides many methods for graphics programming.

Commonly used methods of Graphics class:

1. **public abstract void drawString(String str, int x, int y):** is used to draw the specified string.
2. **public void drawRect(int x, int y, int width, int height):** draws a rectangle with the specified width and height.
3. **public abstract void fillRect(int x, int y, int width, int height):** is used to fill rectangle with the default color and specified width and height.
4. **public abstract void drawOval(int x, int y, int width, int height):** is used to draw oval with the specified width and height.

5. **public abstract void fillOval(int x, int y, int width, int height):** is used to fill oval with the default color and specified width and height.
6. **public abstract void drawLine(int x1, int y1, int x2, int y2):** is used to draw line between the points(x1, y1) and (x2, y2).
7. **public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer):** is used draw the specified image.
8. **public abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used draw a circular or elliptical arc.
9. **public abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used to fill a circular or elliptical arc.
10. **public abstract void setColor(Color c):** is used to set the graphics current color to the specified color.
11. **public abstract void setFont(Font font):** is used to set the graphics current font to the specified font.

Example of Graphics in applet:

```

1. import java.applet.Applet;
2. import java.awt.*;
3.
4. public class GraphicsDemo extends Applet{
5.
6.     public void paint(Graphics g){
7.         g.setColor(Color.red);
8.         g.drawString("Welcome",50, 50);
9.         g.drawLine(20,30,20,300);
10.        g.drawRect(70,100,30,30);
11.        g.fillRect(170,100,30,30);
12.        g.drawOval(70,200,30,30);
13.
14.        g.setColor(Color.pink);
15.        g.fillOval(170,200,30,30);
16.        g.drawArc(90,150,30,30,30,270);
17.        g.fillArc(270,150,30,30,0,180);
18.    }

```

9. }

10. }

myapplet.html

```

1. <html>
2. <body>
3. <applet code="GraphicsDemo.class" width="300" height="300">
4. </applet>
5. </body>
6. </html>

```

A Graphics object encapsulates all state information required for the basic rendering operations that Java supports. State information includes the following properties.

- The Component object on which to draw.
- A translation origin for rendering and clipping coordinates.
- The current clip.
- The current color.
- The current font.
- The current logical pixel operation function.
- The current XOR alternation color

Class declaration

Following is the declaration for **java.awt.Graphics** class:

```

public abstract class Graphics
    extends Object

```

Class constructors

S.N.	Constructor & Description
1	Graphics() () Constructs a new Graphics object.

Class methods

S.N.	Method & Description
1	abstract void clearRect(int x, int y, int width, int height) Clears the specified rectangle by filling it with the background color of the current drawing surface.
2	abstract void clipRect(int x, int y, int width, int height) Intersects the current clip with the specified rectangle.
3	abstract void copyArea(int x, int y, int width, int height, int dx, intdy) Copies an area of the component by a distance specified by dx and dy.
4	abstract Graphics create() Creates a new Graphics object that is a copy of this Graphics object.
5	Graphics create(int x, int y, int width, int height) Creates a new Graphics object based on this Graphics object, but with a new translation and clip area.
6	abstract void dispose() Disposes of this graphics context and releases any system resources that it is using.
7	void draw3DRect(int x, int y, int width, int height, boolean raised) Draws a 3-D highlighted outline of the specified rectangle.
8	abstract void drawArc(int x, int y, int width, int height, intstartAngle, intarcAngle)

	Draws the outline of a circular or elliptical arc covering the specified rectangle.
9	void drawBytes(byte[] data, int offset, int length, int x, int y) Draws the text given by the specified byte array, using this graphics context's current font and color.
10	void drawChars(char[] data, int offset, int length, int x, int y) Draws the text given by the specified character array, using this graphics context's current font and color.
11	abstract boolean drawImage(Image img, int x, int y, Color bgcolor, ImageObserver observer) Draws as much of the specified image as is currently available.
12	abstract boolean drawImage(Image img, int x, int y, ImageObserver observer) Draws as much of the specified image as is currently available.
13	abstract boolean drawImage(Image img, int x, int y, int width, int height, Color bgcolor, ImageObserver observer) Draws as much of the specified image as has already been scaled to fit inside the specified rectangle.
14	abstract boolean drawImage(Image img, int x, int y, int width, int height, ImageObserver observer) Draws as much of the specified image as has already been scaled to fit inside the specified rectangle.
15	abstract boolean drawImage(Image img, int dx1, int dy1, int dx2, int dy2, int sx1, int sy1, int sx2, int sy2, Color bgcolor, ImageObserver observer) Draws as much of the specified area of the specified image as is currently available, scaling it on the fly to fit inside the specified area of the destination

	drawable surface.
16	abstract boolean drawImage(Image img, int dx1, int dy1, int dx2, int dy2, int sx1, int sy1, int sx2, int sy2, ImageObserver observer) Draws as much of the specified area of the specified image as is currently available, scaling it on the fly to fit inside the specified area of the destination drawable surface.
17	abstract void drawLine(int x1, int y1, int x2, int y2) Draws a line, using the current color, between the points (x1, y1) and (x2, y2) in this graphics context's coordinate system.
18	abstract void drawOval(int x, int y, int width, int height) Draws the outline of an oval.
19	abstract void drawPolygon(int[] xPoints, int[] yPoints, int nPoints) Draws a closed polygon defined by arrays of x and y coordinates.
20	void drawPolygon(Polygon p) Draws the outline of a polygon defined by the specified Polygon object.
21	abstract void drawPolyline(int[] xPoints, int[] yPoints, int nPoints) Draws a sequence of connected lines defined by arrays of x and y coordinates.
22	void drawRect(int x, int y, int width, int height) Draws the outline of the specified rectangle.
23	abstract void drawRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight) Draws an outlined round-cornered rectangle using this graphics context's current

	color.
24	abstract void drawString(AttributedCharacterIterator iterator, int x, int y) Renders the text of the specified iterator applying its attributes in accordance with the specification of the TextAttribute class.
25	abstract void drawString(String str, int x, int y) Draws the text given by the specified string, using this graphics context's current font and color.
26	void fill3DRect(int x, int y, int width, int height, boolean raised) Paints a 3-D highlighted rectangle filled with the current color.
27	abstract void fillArc(int x, int y, int width, int height, intstartAngle, intarcAngle) Fills a circular or elliptical arc covering the specified rectangle.
28	abstract void fillOval(int x, int y, int width, int height) Fills an oval bounded by the specified rectangle with the current color.
29	abstract void fillPolygon(int[] xPoints, int[] yPoints, intnPoints) Fills a closed polygon defined by arrays of x and y coordinates.
30	void fillPolygon(Polygon p) Fills the polygon defined by the specified Polygon object with the graphics context's current color.
31	abstract void fillRect(int x, int y, int width, int height) Fills the specified rectangle.

32	abstract void fillRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight) Fills the specified rounded corner rectangle with the current color.
33	void finalize() Disposes of this graphics context once it is no longer referenced.
34	abstract Shape getClip() Gets the current clipping area.
35	abstract Rectangle getClipBounds() Returns the bounding rectangle of the current clipping area.
36	Rectangle getClipBounds(Rectangle r) Returns the bounding rectangle of the current clipping area.
37	Rectangle getClipRect() Deprecated. As of JDK version 1.1, replaced by getClipBounds().
38	abstract Color getColor() Gets this graphics context's current color.
39	abstract Font getFont() Gets the current font.
40	FontMetrics getFontMetrics() Gets the font metrics of the current font.

41	abstract FontMetrics getFontMetrics(Font f) Gets the font metrics for the specified font.
42	boolean hitClip(int x, int y, int width, int height) Returns true if the specified rectangular area might intersect the current clipping area.
43	abstract void setClip(int x, int y, int width, int height) Sets the current clip to the rectangle specified by the given coordinates.
44	abstract void setClip(Shape clip) Sets the current clipping area to an arbitrary clip shape.
45	abstract void setColor(Color c) Sets this graphics context's current color to the specified color.
46	abstract void setFont(Font font) Sets this graphics context's font to the specified font.
47	abstract void setPaintMode() Sets the paint mode of this graphics context to overwrite the destination with this graphics context's current color.
48	abstract void setXORMode(Color c1) Sets the paint mode of this graphics context to alternate between this graphics context's current color and the new specified color.
49	String toString() Returns a String object representing this Graphics object's value.

50

abstract void translate(int x, int y)

Translates the origin of the graphics context to the point (x, y) in the current coordinate system.

Methods inherited

This class inherits methods from the following classes:

- java.lang.Object

Graphics Example

Create the following java program using any editor of your choice in say **D:/ > AWT > com > tutorialspoint > gui >**

AWTGraphicsDemo.java

```
package com.tutorialspoint.gui;

import java.awt.*;
import java.awt.event.*;
import java.awt.geom.*;

public class AWTGraphicsDemo extends Frame {

    public AWTGraphicsDemo() {
        super("Java AWT Examples");
        prepareGUI();
    }

    public static void main(String[] args) {
        AWTGraphicsDemo awtGraphicsDemo = new AWTGraphicsDemo();
        awtGraphicsDemo.setVisible(true);
    }

    private void prepareGUI() {
        setSize(400, 400);
    }
}
```

```
addWindowListener(newWindowAdapter(){  
publicvoid windowClosing(WindowEvent windowEvent){  
    System.exit(0);  
}  
});  
}  
  
@Override  
publicvoid paint(Graphics g){  
    g.setColor(Color.GRAY);  
    Font font=newFont("Serif",Font.PLAIN,24);  
    g.setFont(font);  
    g.drawString("Welcome to TutorialsPoint",50,150);  
}  
}
```

Compile the program using command prompt. Go to **D:/ > AWT** and type the following command.

```
D:\AWT>javac com\tutorialspoint\gui\AWTGraphicsDemo.java
```

If no error comes that means compilation is successful. Run the program using following command.

```
D:\AWT>java com.tutorialspoint.gui.AWTGraphicsDemo
```